

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа 2

Реализация микросервисной архитектуры

Выполнил:
Цатинян Артём
Группа БР1.2

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

1 Задача

Тема проекта: приложение для бронирования столиков в ресторанах.

В рамках лабораторной работы требовалось перейти от монолитного backend-приложения из первой лабораторной работы к практической микросервисной реализации. Необходимо было разделить систему на несколько сервисов, обеспечить принцип **database-per-service**, реализовать межсервисное взаимодействие, сохранить основные пользовательские сценарии бронирования и подготовить демонстрационный набор данных для ручной проверки через Swagger.

2 Ход работы

Исходное приложение из ЛР1 было реализовано как монолитный REST API на Kotlin и Spring Boot. В нем в одной схеме хранились пользователи, рестораны, меню, столики, бронирования и отзывы. Для второй лабораторной работы приложение было вынесено в Gradle multi-module проект, где каждый бизнес-сервис запускается как отдельное Spring Boot приложение.

Реализация сделана на том же технологическом стеке, что и первая лабораторная работа: Kotlin, Spring Boot 3, PostgreSQL, Liquibase XML, jOOQ и Gradle. Для каждого сервиса используется собственная схема базы данных. В локальном окружении все схемы находятся в одной базе **storage_local**, чтобы проект было проще запускать и показывать на защите, но сервисы не используют внешние ключи и SQL-запросы к чужим схемам.

2.1 Состав микросервисов

Сервис	Порт	Ответственность	Схема
identity-service	8081	Регистрация, логин, профиль пользователя, выпуск и проверка JWT	identity
catalog-service	8082	Каталог ресторанов, кухни, меню, столики, рабочие часы	catalog
booking-service	8083	Расчет доступности, создание, просмотр и отмена бронирований	booking

<code>review-service</code>	8084	Просмотр и создание отзывов, проверка права оставить отзыв	<code>review</code>
-----------------------------	------	---	---------------------

В проекте из ДЗ4 целевой архитектурой был также предусмотрен API Gateway. В лабораторной работе отдельный gateway не реализовывался, чтобы не раздувать инфраструктурную часть. Вместо этого каждый сервис имеет свой Swagger UI и публичные REST endpoint-ы, а авторизация и межсервисные проверки реализованы напрямую между сервисами.

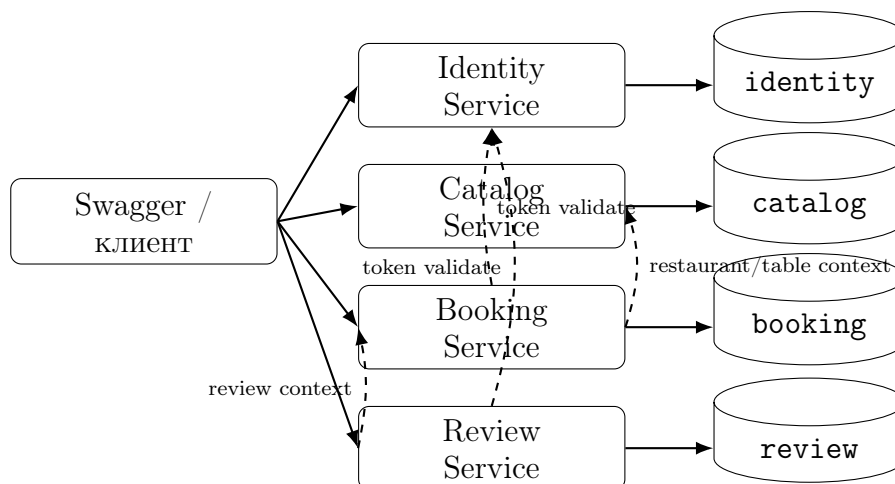
2.2 Структура проекта

Проект расположен в каталоге `labs/lab2`. Общий код вынесен в модуль `common`. В каждом сервисе используется одинаковый подход к структуре:

Пакет	Назначение
<code>adapter/rest</code>	Публичные и внутренние HTTP-контроллеры
<code>adapter/rest/dto</code>	DTO запросов и ответов
<code>adapter/jooq</code>	Репозитории на jOOQ для доступа к собственной схеме сервиса
<code>adapter/client</code>	REST-клиенты для обращения к другим сервисам
<code>service</code>	Бизнес-логика и orchestration сценариев
<code>domain</code>	Доменные записи и enum-ы сервиса
<code>src/generated/jooq</code>	Сгенерированные jOOQ таблицы, records, POJO и DAO

Контроллеры, сервисы, репозитории и клиенты разделены по файлам. Это отличает итоговую реализацию от простого каркаса, где весь сервис мог быть собран в одном классе.

2.3 Архитектурная схема



Сплошные стрелки показывают публичные запросы и работу сервиса со своей базой. Пунктирные стрелки показывают внутренние REST-вызовы между сервисами. Прямых SQL-запросов из одного сервиса в таблицы другого сервиса нет.

2.4 Разделение базы данных

Для локального запуска используется одна PostgreSQL база:

```
database: storage_local
user: storage
password: storage
host: localhost
port: 5432
```

Внутри базы созданы четыре независимые схемы:

Схема	Основные таблицы
identity	users
catalog	restaurants, cuisines, restaurant_cuisines, restaurant_tables, restaurant_working_hours, menu_categories, menu_items, restaurant_photos
booking	bookings
review	reviews, review_outbox

Liquibase changelog-и лежат отдельно в каждом сервисе. При запуске сервис применяет только свои миграции. jOOQ генерирует классы также отдельно для каждой схемы, например таблицы **Users**, **Restaurants**, **Bookings** и **Reviews** находятся в разных пакетах соответствующих сервисов.

2.5 Авторизация

Авторизация реализована через `identity-service`. Сервис предоставляет endpoint-ы регистрации, логина и OAuth2 password token endpoint:

```
POST http://localhost:8081/api/v1/auth/register
POST http://localhost:8081/api/v1/auth/login
POST http://localhost:8081/api/v1/auth/token
```

Swagger UI в остальных сервисах настроен на OAuth2 password flow. Пользователь нажимает **Authorize**, вводит email и пароль, после чего Swagger отправляет запрос в `identity-service`, получает JWT и автоматически прикладывает его как **Authorization: Bearer <token>**.

Проверка токена в сервисах сделана не через передачу объекта пользователя в body запроса, а через общий argument resolver из модуля `common`. Он читает Bearer token из заголовка и вызывает внутренний endpoint `identity-service` для проверки токена и получения текущего пользователя. Для выпуска и проверки JWT используется стандартный механизм Spring Security OAuth2 JOSE: `NimbusJwtEncoder` и `NimbusJwtDecoder`.

2.6 Межсервисные взаимодействия

Кто вызывает	Кого вызывает	Зачем
booking-service	identity-service	Проверить Bearer token и получить текущего пользователя для защищенных ручек бронирований
review-service	identity-service	Проверить Bearer token и получить автора отзыва
booking-service	catalog-service	Получить restaurant/table context: существует ли ресторан, активен ли столик, подходит ли вместимость, какие рабочие часы
review-service	booking-service	Проверить, что бронирование принадлежит пользователю, относится к ресторану и завершено

Таким образом, сервисы не являются четырьмя независимыми CRUD-приложениями. Основные пользовательские операции проходят через цепочки межсервисных вызовов.

2.6.1 Создание бронирования

1. Клиент авторизуется через Swagger и получает JWT от `identity-service`.
2. Клиент вызывает `POST/api/v1/bookings` в `booking-service`.
3. `booking-service` через общий `resolver` обращается в `identity-service` и определяет текущего пользователя.
4. Затем `booking-service` вызывает `catalog-service`, чтобы получить информацию о ресторане и столике.
5. `catalog-service` возвращает название ресторана, номер столика, вместимость, статус и рабочие часы.
6. `booking-service` проверяет пересечения с уже существующими бронированиями в своей БД.
7. Если конфликтов нет, создается запись в схеме `booking`.

В этом сценарии Catalog нужен не для проверки занятости, а для проверки справочной части: ресторан существует, столик принадлежит этому ресторану, столик активен, его вместимость подходит под количество гостей, а выбранное время попадает в рабочие часы. Занятые интервалы проверяются в Booking, потому что только Booking владеет бронированиями.

2.6.2 Получение доступности

1. Клиент вызывает `GET/api/v1/restaurants/{restaurantId}/availability` в `booking-service`.
2. `booking-service` вызывает `catalog-service` и получает список активных столиков и рабочие часы ресторана.
3. Затем `booking-service` читает свои бронирования и исключает занятые интервалы.
4. Клиент получает список доступных столиков и временных интервалов.

2.6.3 Создание отзыва

1. Клиент авторизуется и вызывает `POST/api/v1/restaurants/{restaurantId}/reviews` в `review-service`.

2. `review-service` проверяет токен через `identity-service`.
3. `review-service` вызывает `booking-service` по внутреннему endpoint-y `review context`.
4. `booking-service` проверяет, что бронирование принадлежит пользователю, относится к нужному ресторану и имеет статус `COMPLETED`.
5. `review-service` проверяет, что по этому бронированию еще нет отзыва.
6. Если все проверки пройдены, отзыв сохраняется в схему `review`.

Если пользователь пытается оставить отзыв по будущему или чужому бронированию, сервис возвращает ошибку. Это демонстрирует, что бизнес-правило находится не только внутри Review, а проверяется через владельца бронирований.

2.7 Публичные endpoint-ы

Endpoint	Сервис	Назначение
<code>POST/api/v1/auth/register</code>	Identity	Регистрация пользователя
<code>POST/api/v1/auth/login</code>	Identity	Логин и получение токена
<code>GET/api/v1/users/me</code>	Identity	Получение профиля текущего пользователя
<code>PATCH/api/v1/users/me</code>	Identity	Обновление профиля
<code>GET/api/v1/cuisines</code>	Catalog	Список кухонь
<code>GET/api/v1/restaurants</code>	Catalog	Поиск ресторанов с фильтрацией, сортировкой и пагинацией
<code>GET/api/v1/restaurants/{restaurantId}</code>	Catalog	Детали ресторана
<code>GET/api/v1/restaurants/{restaurantId}/menu</code>	Catalog	Меню ресторана с фильтрацией

GET/api/v1/restaurants/{restaurantId}/availability	Booking	Доступность столиков
POST/api/v1/bookings	Booking	Создание бронирования
GET/api/v1/bookings/me	Booking	Мои бронирования
GET/api/v1/bookings/{bookingId}	Booking	Детали бронирования
PATCH/api/v1/bookings/{bookingId}/cancel	Booking	Отмена бронирования
GET/api/v1/restaurants/{restaurantId}/reviews	Review	Список отзывов ресторана
POST/api/v1/restaurants/{restaurantId}/reviews	Review	Создание отзыва

2.8 Swagger и ручная проверка

После запуска сервисов Swagger UI доступен по адресам:

```
http://localhost:8081/swagger-ui/index.html
http://localhost:8082/swagger-ui/index.html
http://localhost:8083/swagger-ui/index.html
http://localhost:8084/swagger-ui/index.html
```

Для демонстрации используются подготовленные пользователи:

Email	Пароль	Назначение
emily.carter.lab2@example.com	storageAdmin123	Основной пользователь для проверки
mark.taylor.lab2@example.com	storageAdmin123	Второй пользователь для проверки ownership

Важные id тестовых данных:

Сущность	Id	Как используется
North Garden restaurant	1	Ресторан для availability, booking и review сценариев

Pasta Lane restaurant	2	Ресторан для проверки поиска и отзывов
North Garden table T-02	2	Столик для создания новой брони
Completed booking without review	1001	Успешное создание отзыва
Completed booking with existing review	1002	Ошибка повторного отзыва
Future confirmed booking	1003	Отмена бронирования и ошибка отзыва по незавершенной брони

2.9 Сценарии демонстрации

2.9.1 Сценарий 1. Авторизация через Identity из другого сервиса

1. Открыть Swagger booking-service: `http://localhost:8083/swagger-ui/index.html`.
2. Нажать Authorize.
3. Ввести username `emily.carter.lab2@example.com` и password `storageAdmin123`.
4. Swagger отправит запрос на `http://localhost:8081/api/v1/auth/token`.
5. Вызвать `GET /api/v1/bookings/me?page=1&size=10`.

Ожидаемый результат: booking-service вернет бронирования Emily. При этом сам сервис бронирований не принимал пользователя в теле запроса, а достал его из JWT через identity-service.

2.9.2 Сценарий 2. Создание бронирования через Booking и Catalog

Перед созданием бронирования можно вызвать:

```
GET /api/v1/restaurants/1/availability?date=2026-06-12&guestsCount=4
```

Затем выполнить:

```
{
  "restaurantId": 1,
  "tableId": 2,
  "startsAt": "2026-06-12T19:00:00",
  "endsAt": "2026-06-12T21:00:00",
  "guestsCount": 4,
  "specialRequests": "Window table if available"
}
```

Ожидаемый результат: **booking-service** создаст бронирование. Внутри он обратится в **catalog-service**, получит restaurant/table context и только после этого проверит занятость в своей БД.

2.9.3 Сценарий 3. Успешное создание отзыва

В Swagger **review-service** авторизоваться как Emily и вызвать:

```
POST /api/v1/restaurants/1/reviews
```

```
{
  "bookingId": 1001,
  "rating": 5,
  "comment": "Great dinner, fast service and a very comfortable table."
}
```

Ожидаемый результат: 201 Created. **review-service** проверит пользователя через **identity-service**, затем спросит у **booking-service**, можно ли по booking 1001 оставить отзыв.

2.9.4 Сценарий 4. Ошибка при создании отзыва

Для проверки запрета нужно вызвать тот же endpoint, но с незавершенным бронированием:

```
{
  "bookingId": 1003,
  "rating": 4,
  "comment": "Trying to review a future booking."
}
```

Ожидаемый результат: ошибка 422 Unprocessable Entity, потому что booking 1003 еще не завершен. Это правило проверяется в **booking-service**, а не локально в Review.

2.9.5 Сценарий 5. Проверка ownership

1. В Swagger выйти из авторизации.

2. Авторизоваться как `mark.taylor.lab2@example.com`.
3. Вызвать `GET/api/v1/bookings/1003` в `booking-service`.

Ожидаемый результат: `404 Not Found`, потому что `booking 1003` принадлежит Emily. Сервис не раскрывает чужие бронирования другому пользователю.

2.10 Сборка и запуск

Сборка проекта:

```
$env:GRADLE_USER_HOME=(Join-Path (Get-Location) '.gradle-home')
.\gradlew.bat --console=plain assemble
```

Команда применяет Liquibase миграции, запускает jOOQ code generation и компилирует сервисы. PostgreSQL должен быть запущен, потому что jOOQ читает метаданные из базы.

Запуск сервисов:

```
.\gradlew.bat --console=plain :catalog-service:bootRun
.\gradlew.bat --console=plain :identity-service:bootRun
.\gradlew.bat --console=plain :booking-service:bootRun
.\gradlew.bat --console=plain :review-service:bootRun
```

Рекомендуемый порядок запуска: `catalog-service`, `identity-service`, `booking-service`, `review-service`. Это связано с тем, что Booking зависит от Catalog и Identity, а Review зависит от Booking и Identity.

2.11 Подключение к базе данных

Для подключения через `psql`:

```
psql -h localhost -p 5432 -U storage -d storage_local
```

После подключения можно переключаться между схемами:

```
set search_path to identity;
set search_path to catalog;
set search_path to booking;
set search_path to review;
```

3 Вывод

В результате лабораторной работы монолитное приложение бронирования ресторанов было преобразовано в набор микросервисов: Identity, Catalog,

Booking и Review. Каждый сервис получил собственную зону ответственности, собственную схему БД, Liquibase changelog-и и jOOQ-слой доступа к данным.

Основные пользовательские сценарии были сохранены, но теперь выполняются через реальные межсервисные взаимодействия. Booking Service обращается к Catalog Service для получения данных о ресторане и столиках, Review Service обращается к Booking Service для проверки права оставить отзыв, а защищенные endpoint-ы используют Identity Service для проверки JWT. Для демонстрации подготовлены Swagger UI, OAuth2 авторизация, тестовые данные и ручные сценарии проверки.